

密级状态：绝密() 秘密() 内部() 公开(√)

RKAUTH 授权工具使用说明

(技术部，图形显示平台中心)

文件状态： [] 正在修改 [√] 正式发布	当前版本：	V2.0.0
	作 者：	杨华聪
	完成日期：	2024-11-14
	审 核：	熊伟
	完成日期：	2024-11-14

福州瑞芯微电子股份有限公司

Fuzhou Rockchips Semiconductor Co., Ltd

(版本所有,翻版必究)

更新记录

版本	修改人	修改日期	修改说明	核定人
V1.0	杨华聪	2022-10-20	初始版本	熊伟

目 录

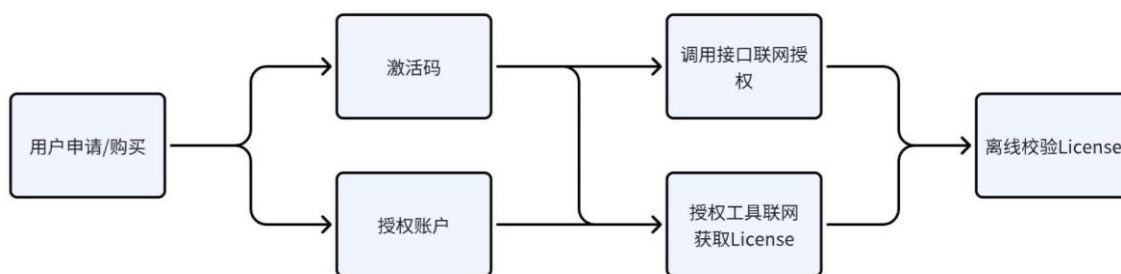
1	功能说明.....	4
2	使用说明.....	4
2.1	授权工具使用流程.....	4
2.2	快速上手使用.....	4
2.2.1	设备上直接授权（设备可联网）	4
2.2.2	PC 上授权（设备无法联网）	5
2.2.3	USB 加密狗授权.....	5
2.2.4	算法 SDK 授权配置.....	7
2.3	授权工具详细说明.....	8
2.3.1	平台和操作系统支持	8
2.3.2	授权工具可执行程序	9
3	量产授权.....	11
4	FAQ 说明.....	13

1 功能说明

RKAUTH 授权工具包用于获取算法授权，使用前客户需要向业务申请算法模块的授权账号密码或激活码。

2 使用说明

2.1 授权工具使用流程



授权工具使用总体流程如下：

- 1) 客户申请/购买算法 SDK 的授权
- 2) 根据申请的算法 SDK 采用的集成方式，提供激活码或授权账户
- 3) 进行激活操作获取 License
 - 激活码：使用接口调用（算法 SDK 提供外部接口，内部调用授权 SDK 的接口）或授权工具
 - 授权账户：使用授权工具，可以在 PC 或设备上操作
- 4) 获取 License 后，后续只需要将 License 传递给算法 SDK 进行离线校验

2.2 快速上手使用

2.2.1 设备上直接授权（设备可联网）

将对应平台的 rkauth_tool_bin 程序，通过 adb 或拷贝方式部署到设备中，然后执行授权命令

```
./rkauth_tool_bin --user="xxxxxx" --passwd="xxxxxx" --output="key.lic" -m module
```

其中-m 指定的 module 为需要授权模块名，请参考算法 SDK 文档填写。

执行成功后授权信息将保存到"key.lic"文件。

2.2.2 PC 上授权（设备无法联网）

如果设备无法联网，可以在 PC 授权，需要通过在设备端执行 rkdevice_info 程序获取设备信息，然后在 PC 端执行 rkauth_tool_bin 程序来获取授权 License。

1) 获取设备信息

将对应平台的 rkdevice_info 执行程序，通过 adb 或拷贝方式部署到设备中，例如 rv1109 平台：

```
adb push bin/Linux/armhf/rkdevice_info /data/  
adb shell  
/data/rkdevice_info /data/device.inf
```

执行成功将在指定的目录生成 device.inf 文件，将该文件拷贝回 PC。

2) PC 获取授权文件

确保 PC 能够联网，使用 bin 目录下面对应平台（Windows/Linux）的可执行程序

```
./rkauth_tool_bin --user="xxxxxx" --passwd="xxxxxx" --output="key.lic" -  
-device_info=/path/to/device.inf -m module
```

其中-m 指定的 module 为需要授权模块名，请参考算法 SDK 文档填写。

执行成功后授权信息保存到 key.lic 文件，将其拷贝到设备中使用。

2.2.3 USB 加密狗授权

USB 加密狗授权方式可以用于无法联网环境，例如产线电脑等。当前 USB 加密狗方式仅 Windows 平台授权工具可以支持。

授权工具运行示例如下：

```
./rkauth_tool_bin -u <vendor> -p <vendor code> -t 2 --output="key.lic" --
device_info=/path/to/device.inf -m module
```

次数查询示例

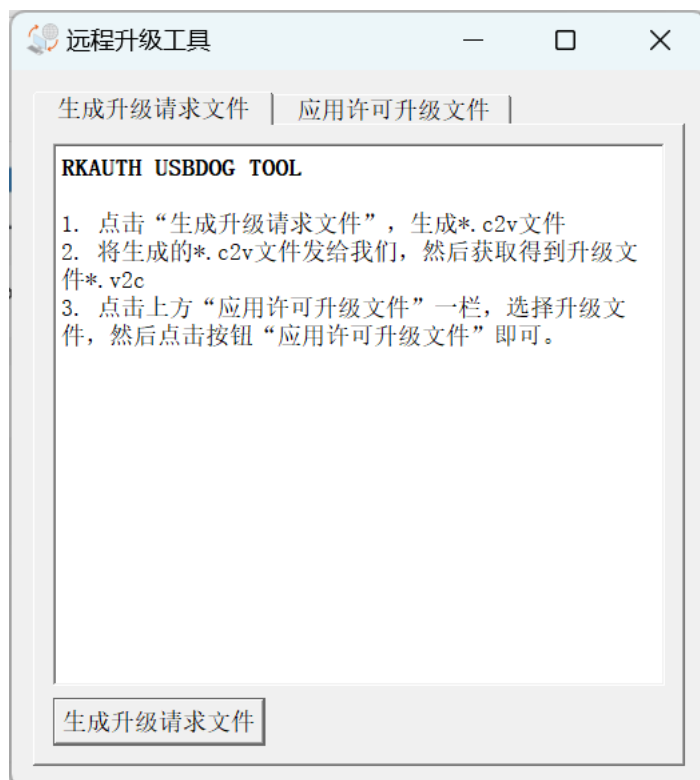
```
./rkauth_tool_bin -u <vendor> -p <vendor code> -t 2 -m module -a 1
```

其中一些参数说明如下：

- **-t 2 参数：**指定使用 USB 加密狗方式进行授权
- **-u 参数：**配置和加密狗同步提供的 vendor 字符串
- **-p 参数：**配置和加密狗同步提供的 vendor code 字符串。

注意

1. USB 加密狗方式不能判断设备是否已授权，请谨慎判断设备是否授权，避免重复授权扣次数
2. 如果加密狗次数用完，可以申请对加密狗次数进行升级，打开“rkauth_tool_usbdog.exe”程序，按照提示进行操作



2.2.4 算法 SDK 授权配置

算法 SDK 集成新版本授权能够支持用户用 JSON 配置选择授权方式（注意，需要算法 SDK 集成了新的版本并提供了可以传递 JSON 配置的方式才可以支持），具体配置接口可以参见各算法 SDK 的说明文档，这里介绍 JSON 配置格式。

用户可以构造 JSON 配置来选择授权的方式，配置格式如下：

```
{
  "trial": 1,
  "rkauth_modules_config": [
    {
      "module": "xxx",
      "activate_code_vendor_storage_id": 99,
      "activate_code_vendor_storage_size": 256,
      "license_vendor_storage_id": 100,
      "license_vendor_storage_size": 1024,
      "activate_code_path": "/path",
      "license_path": "/path"
    }
  ]
}
```

上述并不需要配置所有的字段，只需要配置需要字段即可，其中每个字段表示如下：

- **trial:** 试用，0：关闭试用，如果授权失败直接返回失败，1：开启试用
- **rkauth_modules_config:** 模块列表，存放多个模块的授权配置，每个模块授权配置定义如下：
 - **module:** 授权模块名称
 - **license:** 授权文件字符串内容
 - **license_path:** 授权文件路径
 - **license_vendor_storage_id:** license 存放在 vendor storage 的 item id
 - **activate_code:** 激活码
 - **activate_code_path:** 激活码存放文件路径
 - **activate_code_vendor_storage_id:** 激活码存放 vendor storage 的 item id
 - **user:** 用户名

■ **password:** 密码

注，vendor storage 为设备 flash 上的特殊区域，设计用于存放 SN、MAC 等，可以参考系统 SDK 中文档《Rockchip_Application_Notes_Storage_CN》的“7. Vendor Storage 使用说明”一节。

下面是一些常用配置示例：

1) 使用 License 文件激活

```
{
  "rkauth_modules_config": [{
    "module": "face",
    "license_path": "/sdcard/key.lic"
  },
]
```

其中 license_path 填写对应的 license 文件的路径

2) 使用激活码激活

```
{
  "rkauth_modules_config": [{
    "module": "face",
    "activate_code": "xxx",
    "license_path": "/sdcard/key.lic"
  },
]
```

如果 license_path 配置的 license 路径存在有效文件，会优先离线校验该 license，如果校验失败，将根据配置的激活码请求一个 license，然后存到该路径下。

2.3 授权工具详细说明

授权工具主要用于请求 License 文件。

2.3.1 平台和操作系统支持

	aarch64	armhf	armhf_uclibc	x64	x86
Linux	Y	Y	Y	Y	

Windows				Y	Y
Android	Y	Y			

以下是一些常见平台对应使用的授权工具路径

芯片	授权工具路径
RK3588/RK3576 /RK356X/RK3399Pro	Android: rkauth_tool\bin\Android\arm64-v8a Linux: rkauth\rkauth_tool\bin\Linux\aarch64
RV1109/RV1126	Linux: rkauth\rkauth_tool\bin\Linux\ armhf
RV1106	Linux: rkauth\rkauth_tool\bin\Linux\armhf_uclibc
RK1808/RK3308	Linux: rkauth\rkauth_tool\bin\Linux\aarch64

注：

RV1106 平台 SDK 默认会关闭系统节点，需要内核配置使能后才能支持授权，方法如下：

在 kernel/arch/arm/configs/rv1106_defconfig 中设置 CONFIG_NVMEM_SYSFS=y，

重新编译固件烧写启动后，确认节点是否存在：/sys/bus/nvmem/devices/rockchip-otp0/nvmem

2.3.2 授权工具可执行程序

当前授权工具包包含三个可执行程序：rkauth_tool_bin、rkauth_verify_licence、rkdevice_info（仅设备端支持）。

1) rkauth_tool_bin

用于请求 License，支持激活码或授权账户

命令参数：

```
$ ./rkauth_tool_bin --help
usage: ./rkauth_tool_bin [options] ...
options:
  -u, --user          username(type=0) or activate code(type=1) (string [=])
  -p, --passwd        password (string [=])
  -o, --output        output file (string [=key.lic])
  -d, --device_info   device info (string [=])
  -m, --module        modules to auth(as: face,audio3a,sr,memc...) (string
[=])
  -t, --type          auth type 0: auth account; 1: activate code (int [=0])
  -a, --action        action 0:get licence; 1:show times; 2:get licence string
(int [=0])
  -c, --comment       license file with comment. 0: has comment; 1:no coment
(int [=1])
  -s, --soc           soc chip: 3588/3568/1109/1106 (int [=0])
```

```
-v, --show_version    show version
-?, --help             print this message
```

授权账户运行示例：

```
$ ./rkauth_tool_bin -u <user> -p <password> -d <device info> -m "face"
```

如果是在设备执行，不需要加-d 参数

授权账户查询示例：

可以查询授权账户的已使用次数和总次数

```
$ ./rkauth_tool_bin -u <user> -p <password> -a 1 -m "face"
modules usages:
  face 152/200
```

其中“152/200”对应“已使用次数/总次数”。

激活码授权示例：

```
$ ./rkauth_tool_bin -u <activate code> -t 1 -d <device info> -m "face"
```

2) rkauth_verify_licence

用于校验 License，确保 License 有效

命令参数

```
$ ./rkauth_verify_licence --help
usage: ./rkauth_verify_licence --lic=string [options] ...

return code define:
  0: RKAUTH_OK
  -1: RKAUTH_FAIL
  -2: RKAUTH_LICENSE_INVALID
  -5: RKAUTH_LICENSE_OUTDATE
  -6: RKAUTH_ACCESS_FILE_ERROR

options:
  -l, --lic          license (string)
  -t, --lic_type     license type: 0: file; 1: string; 2: vendor
                    storage id (int [=0])
  -m, --module       modules to verify(like: face,pose) (string
```

```
[=face])  
-?, --help          print this message
```

示例：

```
# ./rkauth_verify_licence -l key.lic  
0
```

3) rkdevice_info

用于在设备上获取设备信息

参数

```
./rkdevice_info <output device info path>
```

示例：

```
# ./rkdevice_info device.inf  
success save device info to device.inf  
# cat device.inf  
3566:a87bf82fad6c2c3b
```

3 量产授权

对量产设备的授权，根据不同的生产环境，可在不同阶段获取 License，分为生产时获取和出厂后获取两种。

获取授权阶段	授权方式	简要说明
设备生产时获取	授权账户	厂测 PC 端请求 License 并写入设备
	授权账户	设备端直接进入厂测模式联网请求授权
	USB 加密狗	厂测 PC 连接 USB 机密狗请求 License 并写入设备
设备出厂后获取	激活码	生产时预写入激活码
	激活码	客户服务器自行分发激活码
	激活码	客户服务器代理授权

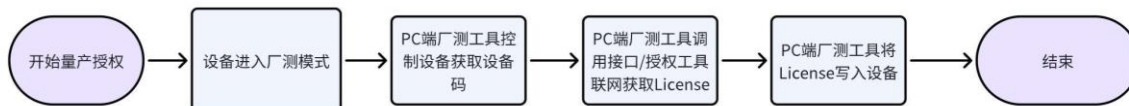
1) 生产时获取 License

生产时获取 License 一般可以在厂测阶段进行，集成到客户原来的厂测工具和程序的流

程中。可以分为以下两种方式：

a) 厂测 PC 端请求 License

适用场景：厂测 PC 可以联网，厂测 PC 和设备可以连接通信（如 IPC/门铃等）



License 可以写入 Vendor Storage 或者不会升级擦除的特殊分区，避免固件升级导致 License 丢失。

b) 设备端请求 License

适用场景：没有厂测 PC，设备端可以联网（有些是生产时临时插 usb 网卡等方式）并有独立厂测模式（如行车记录仪等）。



2) 出厂后获取 License

如果设备在出厂后可以进行联网激活操作（如平板/机顶盒等），也可以考虑使用激活码的方式，在出厂后获取 License。激活码的分发可以分为两种：

a) 生产时预先写入激活码

生产写号阶段预先将激活码写到设备指定位置（vendor storage 或设备分区）：



在产品首次开机联网后，后台进行激活



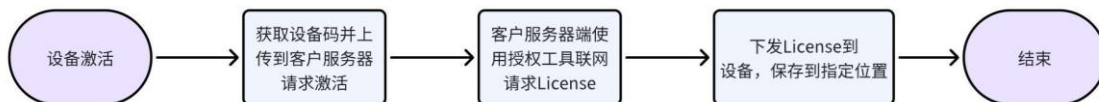
b) 客户服务器自行分发激活码

生产时不需要任何操作，在产品首次开机联网后，后台向客户服务器请求激活码



c) 客户服务器端代理进行激活操作

适用于有些产品设备出于安全考虑不允许访问 RK 授权服务器。设备端可以将设备码发到客户自己服务器进行激活操作，然后再将 License 自行下发到设备。



4 FAQ 说明

- 1) 每个授权次数会绑定一台设备，授权后该设备没有使用次数限制
- 2) 相同的设备重复进行授权不扣次数